

Study RAG Assistant

System specification — architecture, data flows, and feature breakdown, for implementation planning

1. Overview

A study assistant that lets a user upload a document (PDF/DOCX), indexes it for retrieval, auto-generates a topic map on upload, and answers user questions strictly from the document's content — with page-level citations, a confidence/coverage signal per answer, a spaced-repetition flashcard system, and weak-point tracking across study sessions. If an answer is not supported by the document, the system says so explicitly rather than using outside knowledge.

2. Architecture

Core components:

- **Frontend** — upload UI, chat UI, flashcard/review UI, progress dashboard
- **Backend API** — orchestrates ingestion and query pipelines; stateless request handling
- **Postgres + pgvector** — stores document chunks, embeddings, page/paragraph metadata, flashcards, quiz attempt logs
- **LLM API** (Claude/OpenAI) — used for embeddings, topic-map generation, answer generation, flashcard generation, open-answer grading
- **Spaced-repetition scheduler** — pure application logic (no LLM); computes next review date per flashcard

Connection pattern: Frontend → Backend API → (Postgres/pgvector for storage + retrieval) and (LLM API for embeddings + generation). The backend is the only component that talks to the LLM API and the database directly.

3. Ingestion flow (on document upload)

1. Upload document — accept PDF or DOCX file from the user
2. Extract and chunk text — parse the file, split into sections (by heading where possible), tag each chunk with its source page/paragraph number
3. Generate embeddings — call embedding API per chunk
4. Store in vector DB — write chunk text, embedding vector, and page metadata to Postgres/pgvector
5. Generate overview — one LLM call over the full document (or chunk summaries) producing a topic map / table of contents, stored alongside the document record

4. Query flow (on each user question)

1. User asks a question — typed into chat UI, scoped to one uploaded document (or a set, for multi-doc mode)
2. Embed the question — using the same embedding model as ingestion

3. Vector search — retrieve top-k relevant chunks by similarity, each carrying its page metadata and similarity score
4. Build prompt — assemble retrieved chunks + their source labels + the user's question into the LLM prompt, with an explicit instruction to answer only from provided context
5. LLM generates answer — returns the answer text plus which chunk ID(s) it drew from; if no chunk is relevant, it returns an explicit 'not covered in this document' response

5. Feature breakdown — LLM vs. code

General rule: the LLM handles generation and judgment on text (summarizing, answering, grading open-ended responses, phrasing suggestions). Code handles anything requiring determinism or auditability — scores, thresholds, scheduling, counting, and storage/retrieval of facts.

5.1 Structure-aware output (topic map / TOC) — *LLM-driven*

One LLM call on the full document (or on chunk summaries) at upload time, prompted to list the sections/topics covered. Code triggers this once per upload and stores the result for display; no ongoing LLM involvement.

5.2 Citations / traceability — *Code-driven*

At chunking time, code tags every chunk with its page/paragraph number as metadata. That metadata travels with the chunk through retrieval. The LLM only needs to report which chunk ID(s) it used in its answer — code maps chunk ID → page number for display. Page numbers should never be left to the LLM to recall from memory.

5.3 Confidence / coverage signal — *Code-driven (LLM optional, secondary)*

Vector search returns a similarity score per retrieved chunk. Code applies thresholds (e.g. score > 0.75 = "fully covered", 0.5–0.75 = "partially covered", below = "not in document") to assign the label shown to the user. An LLM self-rated confidence can be added as a secondary signal, but the retrieval score should remain the source of truth since LLM self-rating alone is unreliable.

5.4 Spaced repetition — *Code-driven (LLM used once, upfront)*

The LLM is used only once per document to generate flashcard question/answer pairs from the content. The review scheduling itself (e.g. an SM-2 / Anki-style algorithm) is pure deterministic logic based on past review dates and correctness — no LLM call at review time.

5.5 Weak-point detection — *Code-driven (LLM optional, for phrasing only)*

Every question asked and whether the user's answer was marked correct or incorrect is logged, tagged by document section/topic. Code tallies error rate per topic and surfaces a suggestion once a topic's error rate crosses a threshold (e.g. "you're weak on section 4"). An LLM can be used only to phrase the suggestion in natural language — the detection logic itself is counting, not generation.

6. Data model sketch

Table	Key fields
documents	id, title, uploaded_at, topic_map (json)
chunks	id, document_id, text, page_number, embedding (vector)

flashcards	id, document_id, question, answer, section_tag
review_log	id, flashcard_id, reviewed_at, correct (bool), next_review_at
question_log	id, document_id, question, matched_chunk_ids, coverage_label, section_tag, correct (nullable)

7. Suggested stack

- Backend: Python + FastAPI
- Vector storage: Postgres + pgvector
- LLM/embeddings: OpenAI or Claude API
- Orchestration: LangChain or LlamaIndex (optional — can also be hand-rolled)
- Deployment: Docker container, exposed as a REST API